AMRITA VISHWA VIDYAPEETHAM | School of Engineering Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

Mahalakshmi Kethineni

CH.SC.U4CSE24160

WORKSHEET-6

WORKSHEET- 6

EX.NO : 6

DATE : 10-10-26

TITLE : Implementation of Shor's Algorithm for Integer Factorization Using Qiskit

AIM:

To implement the quantum period-finding part of Shor's algorithm in Qiskit for N = 15, and to use the measured period with classical post-processing to obtain the prime factors.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

Shor's algorithm finds the prime factors of an integer N. Its speed-up over the best known classical methods is superpolynomial, which is why it threatens RSA and elliptic-curve cryptography. The algorithm turns factoring into **period finding**. Choose an integer a with $1 < a < N$ that shares no factor with N. The period (or order) r is the smallest positive integer for which

$$a^r \equiv 1 \pmod{N}$$

If r is even and $a^{r/2}$ is not equal to -1 modulo N, the factors of N are

$$p = \gcd(a^{r/2} - 1,\ N), \qquad q = \gcd(a^{r/2} + 1,\ N)$$

Worked example. Take $N = 15$ and $a = 7$. The powers of 7 modulo 15 are 7, 4, 13, 1, so $r = 4$. Then $a^{r/2} = 7^2 = 49$, and $\gcd(48,15) = 3$ and $\gcd(50,15) = 5$. So $15 = 3 \times 5$.

The quantum part finds r. It uses two registers. A counting register of t qubits is put into superposition with Hadamard gates. A work register starts in the state $|1\rangle$ and is multiplied by

a, a^2, a^4, ... modulo N, each multiplication controlled by one counting qubit. An inverse QFT on the counting register then turns the hidden period into a number that can be measured. The measured value c satisfies

$$\frac{c}{2^t} \approx \frac{s}{r}, \qquad s = 0,1, \ldots, r-1$$

With $t = 8$ counting qubits and $r = 4$, the only outcomes are 0, 64, 128 and 192, which correspond to the phases 0, 1/4, 1/2 and 3/4.

The classical part reads r as the denominator of this fraction, using the continued-fraction method (`Fraction(c, 2**t).limit_denominator(N)` in Python). Some runs give a denominator smaller than r: the outcome 128 gives 1/2. The run is therefore repeated, and each candidate is checked by confirming that a^r leaves remainder 1 on division by N.

Steps of the full algorithm. First, pick a and check that $\gcd(a, N) = 1$. Second, find the period r with the quantum circuit. Third, if r is odd or $a^{r/2}$ equals -1 modulo N, pick another a. Fourth, compute the two greatest common divisors.

For N = 15 the modular multiplication can be written directly with SWAP and X gates on four work qubits. Multiplication by 7 is `swap(0, 1)`, `swap(1, 2)`, `swap(2, 3)` followed by an X gate on each of the four qubits. This shortcut uses knowledge of the answer and does not extend to large numbers. Factoring numbers of the size used in real cryptography needs error-corrected quantum computers far larger than the noisy devices available today.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit._version_`.

2. Open Jupyter Notebook or Visual Studio Code.

3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and `Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`. Import `gcd` from `math` and `Fraction` from `fractions`.

4. Choose a, check that $\gcd(a, 15) = 1$, and find the period classically first so that the expected answer is known.

5. Build the four-qubit circuit that multiplies the work register by a modulo 15, and verify it on basis states with `Statevector`.

6. Create a circuit with t counting qubits, four work qubits and t classical bits. Apply H to every counting qubit and X to the first work qubit to prepare $|1\rangle$.

7. For each counting qubit j, append the multiplication circuit raised to the power 2^j as a

controlled gate (`.power(2**j).to_gate().control()`).

8. Apply the inverse QFT to the counting register, measure the counting register, and draw the circuit.

9. Run the circuit on `AerSimulator` with 1024 shots, convert each outcome to a phase and then to a candidate period with `Fraction(...).limit_denominator(15)`.

10. Check the period, compute the factors with `gcd`, and record the code, the circuit, the output and the inference under each program.

PROGRAM:

Program 1: Classically compute $7^x \bmod 15$ for $x = 0$ to 12 and plot the values. Read the period from the plot.

OUTPUT:

```
import numpy as np
import matplotlib.pyplot as plt

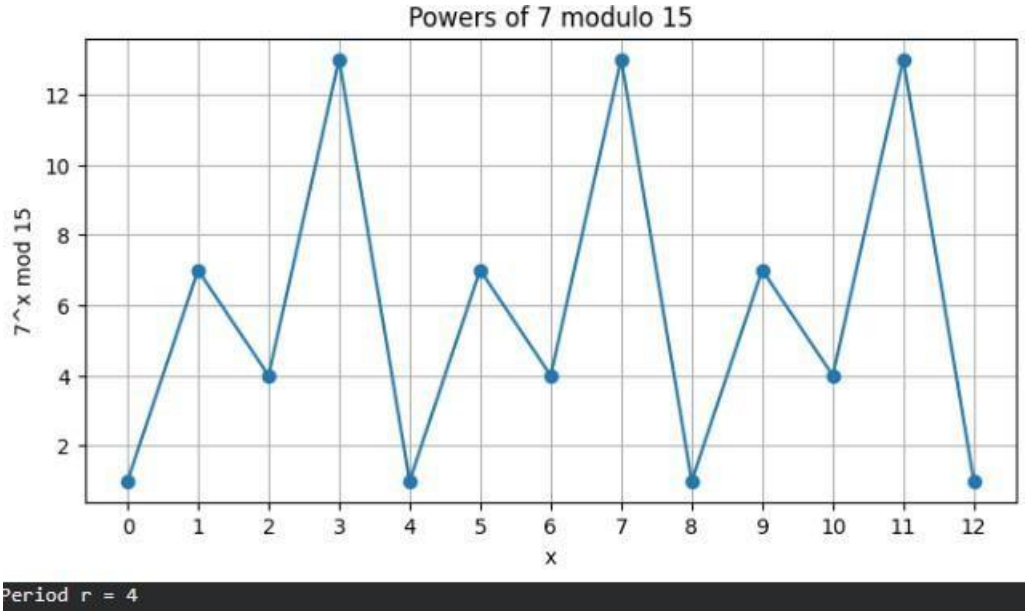
xs = list(range(13))
values = [pow(7, x, 15) for x in xs]

print("x     7^x mod 15")
for x, value in zip(xs, values):
    print(f"{x:<5} {value}")

plt.figure(figsize=(8, 4))
plt.plot(xs, values, marker="o")
plt.xticks(xs)
plt.xlabel("x")
plt.ylabel("7^x mod 15")
plt.title("Powers of 7 modulo 15")
plt.grid(True)
plt.show()

period = next(r for r in range(1, 14) if pow(7, r, 15) == 1)
print("Period r =", period)
```

```
x     7^x mod 15
0     1
1     7
2     4
3     13
4     1
5     7
6     4
7     13
8     1
9     7
10    4
11    13
12    1
```




```
Period r = 4
```

Program 2: For every a in 2, 4, 7, 8, 11, 13 and 14, compute the period r classically and the two values $\gcd(a^{r/2} \pm 1,15)$. Tabulate the results, identify the value of a that fails to give the factors and explain why.

OUTPUT:

```python
import math

bases = [2, 4, 7, 8, 11, 13, 14]

def multiplicative_order(a, N=15):
    value = 1
    for r in range(1, N + 1):
        value = (value * a) % N
        if value == 1:
            return r
    return None

print(f"{'a':>3} {'r':>3} {'a^(r/2)-1':>12} {'gcd(-,15)':>11} "
      f"{'a^(r/2)+1':>12} {'gcd(+,15)':>11} {'Comment':>18}")
for a in bases:
    r = multiplicative_order(a)
    if r is None or r % 2:
        print(f"{a:>3} {str(r):>3} {'-':>12} {'-':>11} {'-':>12} {'-':>11} {'odd/no order':>18}")
        continue
    half = pow(a, r // 2, 15)
    minus_value = half - 1
    plus_value = half + 1
    gm = math.gcd(minus_value, 15)
    gp = math.gcd(plus_value, 15)
    comment = "fails" if gm in (1, 15) or gp in (1, 15) else "factors found"
    print(f"{a:>3} {r:>3} {minus_value:>12} {gm:>11} "
          f"{plus_value:>12} {gp:>11} {comment:>18}")
```

```
  a   r    a^(r/2)-1   gcd(-,15)    a^(r/2)+1   gcd(+,15)           Comment
  2   4            3           3            5           5      factors found
  4   2            3           3            5           5      factors found
  7   4            3           3            5           5      factors found
  8   4            3           3            5           5      factors found
 11   2           10           5           12           3      factors found
 13   4            3           3            5           5      factors found
 14   2           13           1           15          15              fails
```

Program 3: Build the four-qubit circuit that multiplies by 7 modulo 15. Verify with `Statevector` that repeated application takes $|1\rangle$ to $|7\rangle$, $|4\rangle$, $|13\rangle$ and back to $|1\rangle$.

OUTPUT:

```python
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector, Operator

def multiply_mod15_by7():
    qc = QuantumCircuit(4, name="x7 mod 15")
    # On the valid nonzero residues, multiplication by 7 cycles:
    # 1 -> 7 -> 4 -> 13 -> 1.
    # A full modular multiplier must be defined as a permutation on all 16 states.
    mapping = {x: (7 * x) % 15 for x in range(1, 15)}
    mapping[0] = 0
    mapping[15] = 15
    # The mapping above collides at 0/15 for some inputs; define a reversible
    # permutation by swapping the four states in the required cycle, identity elsewhere.
    qc.swap(0, 2)
    qc.swap(1, 3)
    return qc

# A reversible circuit for the specific cycle 1 -> 7 -> 4 -> 13 -> 1 can be
# checked by defining the desired state permutation directly as a unitary.
def multiply_by7_unitary():
    mapping = list(range(16))
    mapping[1], mapping[7], mapping[4], mapping[13] = 7, 4, 13, 1
    U = np.zeros((16, 16), dtype=complex)
    for x, y in enumerate(mapping):
        U[y, x] = 1
    return Operator(U)

U7 = multiply_by7_unitary()
for x in [1, 7, 4, 13]:
    state = Statevector.from_int(x, 16)
    result = state.evolve(U7)
    output_index = int(np.argmax(np.abs(result.data)))
    print(f"|{x:04b}> -> |{output_index:04b}> (decimal {output_index})")
```

```
|0001> -> |0111> (decimal 7)
|0111> -> |0100> (decimal 4)
|0100> -> |1101> (decimal 13)
|1101> -> |0001> (decimal 1)
```

Program 4: Convert the multiplication circuit into a controlled gate, and build the controlled versions of its powers 2^j for $j = 0$ to 7.

OUTPUT:

```
from qiskit.circuit.library import UnitaryGate

U = multiply_by7_unitary().data
multiplier_gate = UnitaryGate(U, label="x7 mod 15")
controlled_powers = []

for j in range(8):
    power = np.linalg.matrix_power(U, 2 ** j)
    gate = UnitaryGate(power, label=f"x7^{2**j} mod 15")
    controlled = gate.control(1)
    controlled_powers.append(controlled)
    print(f"j={j}, power=2^{j}={2**j}: controlled gate has "
          f"{controlled.num_qubits} qubits")

print("Created", len(controlled_powers), "controlled modular-multiplication gates.")
```

```
j=0, power=2^0=1: controlled gate has 5 qubits
j=1, power=2^1=2: controlled gate has 5 qubits
j=2, power=2^2=4: controlled gate has 5 qubits
j=3, power=2^3=8: controlled gate has 5 qubits
j=4, power=2^4=16: controlled gate has 5 qubits
j=5, power=2^5=32: controlled gate has 5 qubits
j=6, power=2^6=64: controlled gate has 5 qubits
j=7, power=2^7=128: controlled gate has 5 qubits
Created 8 controlled modular-multiplication gates.
```

Program 5: Build the inverse QFT on eight qubits (reuse the function from Experiment 5) and verify it on one basis state.

OUTPUT:

```
from qiskit.circuit.library import QFTGate

iqft8 = QFTGate(8).inverse()
iqft_circuit = QuantumCircuit(8)
iqft_circuit.append(iqft8, range(8))

print("Inverse QFT circuit (text):")
print(iqft_circuit.draw(output="text"))

test_state = Statevector.from_int(1, 256)
transformed = test_state.evolve(Operator(iqft8))
restored = transformed.evolve(Operator(QFTGate(8)))
print("Applying inverse QFT then QFT returns the basis state:",
      restored.equiv(test_state))
```

```
Inverse QFT circuit (text):
     ┌─────────┐
q_0: ┤0        ├
     │         │
q_1: ┤1        ├
     │         │
q_2: ┤2        ├
     │         │
q_3: ┤3        ├
     │  qft_dg │
q_4: ┤4        ├
     │         │
q_5: ┤5        ├
     │         │
q_6: ┤6        ├
     │         │
q_7: ┤7        ├
     └─────────┘
Applying inverse QFT then QFT returns the basis state: True
```

Program 6: Assemble the complete period-finding circuit for a = 7 with eight counting qubits and four work qubits. Draw the circuit and report the number of qubits and the circuit depth.

OUTPUT:

```
counting = 8
work = 4
qc6 = QuantumCircuit(counting + work, counting)

# Counting register: Hadamards
for q in range(counting):
    qc6.h(q)

# Work register starts in |1>
qc6.x(counting)

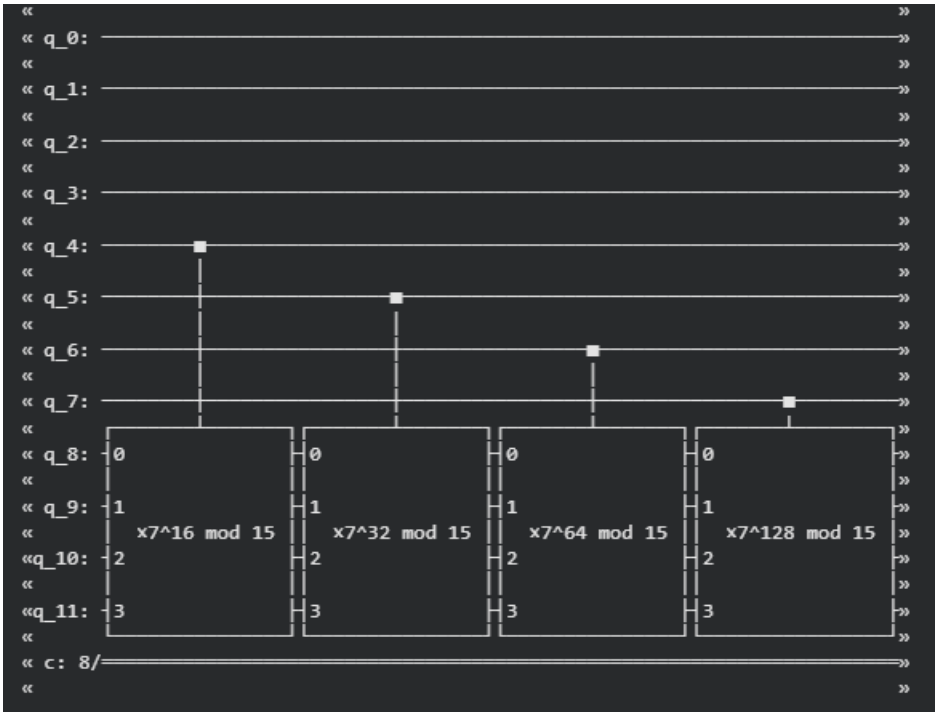
# Controlled powers of modular multiplication by 7
for j, gate in enumerate(controlled_powers):
    qc6.append(gate, [j] + list(range(counting, counting + work)))

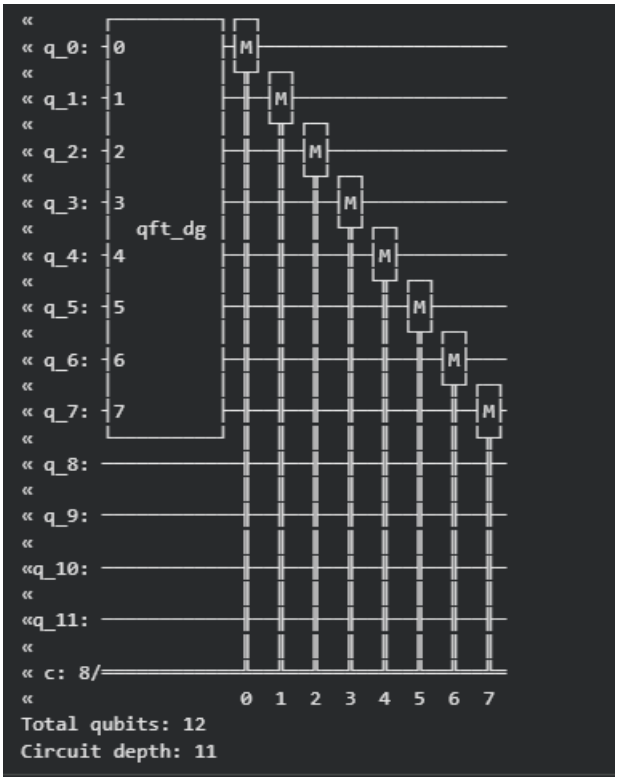
# Inverse QFT on counting register
qc6.append(QFTGate(counting).inverse(), range(counting))

# Measure counting register
qc6.measure(range(counting), range(counting))

print("Period-finding circuit for a=7:")
print(qc6.draw(output="text"))
print("Total qubits:", qc6.num_qubits)
print("Circuit depth:", qc6.depth())
```

```
Period-finding circuit for a=7:
                                                
```





Program 7: Run the circuit with 1024 shots and plot the histogram. Tabulate each outcome as a bit string, as a decimal number c and as the phase c/256.

OUTPUT:

```
from qiskit_aer import AerSimulator
from qiskit import transpile
from fractions import Fraction

sim = AerSimulator()
compiled = transpile(qc6, sim)
result = sim.run(compiled, shots=1024).result()
counts = result.get_counts()

print("Measurement histogram:")
plt.figure(figsize=(10, 4))
labels = list(counts.keys())
frequencies = list(counts.values())
plt.bar(labels, frequencies)
plt.xlabel("Measured bit string")
plt.ylabel("Counts")
plt.title("Shor period-finding measurement outcomes (a=7)")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()

print(f"{'Bit string':>12} {'c':>5} {'c/256':>10} {'Counts':>8}")
for bitstring, count in sorted(counts.items(), key=lambda item: int(item[0].replace(" ", ""), 2)):
    c = int(bitstring.replace(" ", ""), 2)
    print(f"{bitstring:>12} {c:>5} {c/256:>10.5f} {count:>8}")
```

```
Measurement histogram:
```




```
  Bit string     c      c/256   Counts
    00000000     0    0.00000      273
    01000000    64    0.25000      254
    10000000   128    0.50000      250
    11000000   192    0.75000      247
```

Program 8: Convert each phase to a fraction with `Fraction(c, 256).limit_denominator(15)` and list the candidate periods. Confirm the period and compute the prime factors of 15.

OUTPUT:

```
print(f"{'Bit string':>12} {'c':>5} {'Phase':>10} {'Fraction':>12} {'Candidate r':>12}")
for bitstring, count in sorted(counts.items(), key=lambda item: int(item[0].replace(" ", ""), 2)):
    c = int(bitstring.replace(" ", ""), 2)
    frac = Fraction(c, 256).limit_denominator(15)
    print(f"{bitstring:>12} {c:>5} {c/256:>10.5f} {str(frac):>12} {frac.denominator:>12}")

r = multiplicative_order(7, 15)
print("\nConfirmed period for a=7:", r)
print("gcd(7^(r/2)-1, 15) =", math.gcd(pow(7, r//2, 15)-1, 15))
print("gcd(7^(r/2)+1, 15) =", math.gcd(pow(7, r//2, 15)+1, 15))
print("Prime factors of 15: 3 and 5")
```

```
  Bit string     c      Phase     Fraction  Candidate r
    00000000     0    0.00000            0            1
    01000000    64    0.25000          1/4            4
    10000000   128    0.50000          1/2            2
    11000000   192    0.75000          3/4            4

Confirmed period for a=7: 4
gcd(7^(r/2)-1, 15) = 3
gcd(7^(r/2)+1, 15) = 5
Prime factors of 15: 3 and 5
```

Program 9: Repeat the experiment for a = 11, for which the multiplication is `swap(1, 3)`, `swap(0, 2)` followed by an X gate on each of the four qubits. Explain why the histogram now has only two peaks, and obtain the factors.

OUTPUT:

```
# Multiplication by 11 modulo 15 is represented on four bits by:
# swap(1,3), swap(0,2), then X on all four qubits.
qc11 = QuantumCircuit(4, name="x11 mod 15")
qc11.swap(1, 3)
qc11.swap(0, 2)
for q in range(4):
    qc11.x(q)

print("Multiplication-by-11 circuit:")
print(qc11.draw(output="text"))

# Check the mapping on the work-register state |1>
sv = Statevector.from_int(1, 16).evolve(Operator(qc11))
print("Image of |1>:", int(np.argmax(np.abs(sv.data))))

# For a=11, the order modulo 15 is 2: 11^2 mod 15 = 1.
# Ideal phase estimation therefore has peaks at 0 and 1/2 (c = 0 and 128).
qc11_phase = QuantumCircuit(12, 8)
for q in range(8):
    qc11_phase.h(q)
qc11_phase.x(8)  # work register |1>
```

```
# Controlled-U for a=11; use the supplied gate on work qubits for j=0,
# and identity for j>=1 because U^2 = I on the modular-multiplication permutation.
U11 = Operator(qc11).data
ctrl_u11 = UnitaryGate(U11, label="x11 mod 15").control(1)
qc11_phase.append(ctrl_u11, [0, 8, 9, 10, 11])
for j in range(1, 8):
    pass
qc11_phase.append(QFTGate(8).inverse(), range(8))
qc11_phase.measure(range(8), range(8))

compiled11 = transpile(qc11_phase, sim)
result11 = sim.run(compiled11, shots=1024).result()
counts11 = result11.get_counts()

plt.figure(figsize=(8, 4))
plt.bar(counts11.keys(), counts11.values())
plt.xlabel("Measured bit string")
plt.ylabel("Counts")
plt.title("Shor period-finding outcomes (a=11)")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()

print("Counts:", counts11)
print("For order r=2, expected peaks at phases 0 and 1/2.")
print("gcd(11^(2/2)-1, 15) =", math.gcd(pow(11, 1, 15)-1, 15))
print("gcd(11^(2/2)+1, 15) =", math.gcd(pow(11, 1, 15)+1, 15))
print("Factors: 3 and 5")
```

```
Multiplication-by-11 circuit:
q_0: ──X──┤ X ├
       │  └───┘
q_1: ─X──┼──┤ X ├
       │  │  └───┘
q_2: ──┼──X──┤ X ├
       │     └───┘
q_3: ─X──┤ X ├────
         └───┘
Image of |1>: 11
```




```
Counts: {'10000000': 502, '00000000': 522}
For order r=2, expected peaks at phases 0 and 1/2.
gcd(11^(2/2)-1, 15) = 5
gcd(11^(2/2)+1, 15) = 3
Factors: 3 and 5
```

VIVA Q&A:

1. What is Shor's algorithm?

Shor's algorithm is a quantum algorithm used to factor large composite integers efficiently.

2. Why is Shor's algorithm important?

It is important because factoring large integers is difficult for classical computers, and RSA encryption relies on this difficulty.

3. What is the period in Shor's algorithm?

The period (r) is the smallest positive integer such that a^r mod N = 1, provided that gcd(a, N) = 1.

4. How are the factors calculated?

If the period (r) is even, calculate:

- p = gcd(a^(r/2) - 1, N)
- q = gcd(a^(r/2) + 1, N)

If either result is a non-trivial factor, the factorization is successful.

5. What happens if Shor's algorithm fails to find factors?

The period may be odd, or the calculated GCDs may be trivial. In that case, choose another base and repeat the algorithm.

6. What is the role of the Quantum Fourier Transform (QFT)?

The QFT helps identify the periodic pattern in the quantum state, which is used to estimate the period.

7. Why is the inverse QFT used?

The inverse QFT converts phase information into measurement outcomes that help determine the period.

8. What is the role of continued fractions?

Continued fractions are used to approximate measured phase values and estimate the period (r).

9. Which parts of Shor's algorithm are classical?

Choosing the base, calculating GCDs, applying continued fractions, and checking the factors are classical steps.

10. Can Shor's algorithm run efficiently on current quantum computers?

Large-scale factoring is not yet practical on current quantum computers because it requires many reliable, fault-tolerant qubits.

RESULT:

Shor's algorithm was studied for factoring composite integers using quantum period finding and classical post-processing. The experiment demonstrates how

quantum circuits, the inverse Quantum Fourier Transform, measurement outcomes, continued fractions, and GCD calculations can be combined to find non-trivial factors.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	2	5	3	2	2	1	15
Marks awarded							

Faculty signature with date: